

Reviewer Pack — Arithmetic Rank of Quaternion Algebras Bounds Randomized Com...

Entry 54d5df2e4f94 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 00:26:36 UTC

Arithmetic Rank of Quaternion Algebras Bounds Randomized Communication Complexity for Disjointness

Entry ID: 54d5df2e4f94 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 00:26:36 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quaternion Algebras) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘The arithmetic rank of the quaternion algebra associated with an n -ary Boolean function f bounds the randomized communication complexity of the partial function defined by f , such that for all n , if $R_QUAT(f) = \Omega(n)$, then $CC_R(DISJ_n) \geq \Omega(n)$.’, ‘Equivalently, there exists a polynomial-time computable mapping from each n -ary Boolean function to its corresponding quaternion algebra Q_f with $R_QUAT(Q_f) = \Omega(n)$ such that the randomized communication complexity of the disjointness problem on n variables is at least as large as $\Omega(n)$.’, ‘Conversely, if $CC_R(DISJ_n) < \Omega(n)$, then there exists an n -ary Boolean function f such that $R_QUAT(Q_f) \leq \Theta(\log n)$.’]

Rationale (proposer’s reasoning):

[‘Quaternion algebras are non-commutative generalizations of complex numbers and have been studied in various mathematical contexts. Their arithmetic rank has connections to the complexity of certain problems, such as finding solutions to polynomial equations.’, ‘The proposed conjecture suggests that the arithmetic rank of quaternion algebras could provide a lower bound on the communication complexity of the disjointness problem, which is an open question in communication complexity theory.’, ‘This bridge between algebra and complexity theory might expose new structures and insights into the inherent difficulty of problems like disjointness.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 02ceff31f6372be2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of n-ary Boolean functions tested ($n \leq 40$), the arithmetic rank $R_QUAT(Q_f)$ exceeds $\Omega(n)$ and the randomized communication complexity $CC_R(DISJ_n)$ meets or exceeds $\Omega(n)$, with p-value ≤ 0.05 using a suitable statistical test.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'arithmetic rank quaternion algebras' AND 'randomized communication complexity disjointness' - 'quaternion algebra associated boolean function' AND 'partial function communication complexity' - 'polynomial-time computable mapping' AND 'disjointness problem randomized communication complexity'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def quaternion_algebra(f):
        n = len(f)
        if n <= 1:
            return None
        Q_f = []
        for i in range(n):
            row = []
            for j in range(n):
                if f[i] == f[j]:
                    row.append(0)
                else:
                    row.append(1)
            Q_f.append(row)
```

```

    return Q_f

def arithmetic_rank(Q):
    n = len(Q)
    rank = 0
    for i in range(n):
        pivot_row = None
        for j in range(i, n):
            if Q[j][i] != 0:
                pivot_row = j
                break
        if pivot_row is None:
            continue
        rank += 1
        for j in range(n):
            if j == pivot_row:
                continue
            factor = Fraction(Q[j][i], Q[pivot_row][i])
            for k in range(n):
                Q[j][k] -= factor * Q[pivot_row][k]
    return rank

def communication_complexity(f):
    n = len(f)
    if n == 1:
        return 0
    return math.ceil(math.log2(n))

f = generate_random_boolean_function(40)
Q_f = quaternion_algebra(f)
if Q_f is None:
    return {
        "metric_name": "arithmetic_rank",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

R_QUAT = arithmetic_rank(Q_f)
CC_R_DISJ = communication_complexity(f)

return {
    "metric_name": "arithmetic_rank",
    "metric_value": R_QUAT,
    "instances_tested": 1,
    "conjecture_holds": R_QUAT >= math.ceil(0.5 * len(f)),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["metric_value"] is not None for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
else:
    print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the expected time frame. | next: Re-run the test with increased time limits or consider alternative methods to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12710
2	propose	ollama_remote	glm4:latest	0	0	12521
3	preregistration	ollama_remote	glm4:latest	0	0	6483
4	novelty	ollama_remote	glm4:latest	0	0	4741
5	novelty	ollama_remote	glm4:latest	0	0	5257
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15966
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11035
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10224
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9316
10	judge	ollama_remote	glm4:latest	0	0	17358

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105611 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/54d5df2e4f94.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/54d5df2e4f94.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/54d5df2e4f94.tar.gz (if generated)